SKYLITE / SKYNET – Full Technical Manual

Table of Contents

1. **Introduction**
2. **Core Architectural Principles**
3. **System Components Overview**
4. **Request Processing Model**
5. **Page Resolution Engine**
6. **WebPage Lifecycle (Deep Dive)**
7. **HtmlDocument Architecture**
8. **ApiRequest / ApiResponse Model**
9. **JSCore Execution Engine**
10. **Translation System**
11. **Security Architecture**
12. **Error Handling & Diagnostics**
13. **Performance Characteristics**
14. **Naming Conventions & Module Structure**
15. **Template System (HTML/TXT)**
16. **Middleware Architecture (SKYNET)**
17. **Static Environment Compatibility (SKYLITE → SKYNET)**
18. **Database Interaction Patterns**
19. **File Upload/Download Pipeline**
20. **Grid & Form Architecture**
21. **Developer Guidelines & Best Practices**
22. **Extensibility Model**

23. **Appendix A – ASCII Architecture Diagrams**

24. **Appendix B – Lifecycle Sequence Charts**

25. **Appendix C – Example Page Implementations**

1. Introduction

This manual provides a complete, developer-focused technical reference for the SKYLITE (IIS) and SKYNET (ASP.NET Core) frameworks. It describes the architecture, request lifecycle, rendering model, translation engine, security hooks, and extensibility points in maximum detail.

The goal is to give developers a deep understanding of how the framework behaves internally and how to build applications efficiently and correctly.

2. Core Architectural Principles

SKYLITE/SKYNET are built on several foundational principles:

2.1 Server-Driven Rendering

The server generates the full HTML document. The browser acts as a thin client.

2.2 Reflection-Based Routing

Routing is determined by class names, not configuration.

2.3 Deterministic Lifecycle

Every page follows the same GET/POST lifecycle.

2.4 Minimal Client-Side Logic

JSCore executes only framework-defined commands.

2.5 Strong Separation of Concerns

- Server: logic, rendering, translation
- Browser: DOM updates, event dispatch

2.6 Predictability

The framework avoids “magic” and hidden behavior.

3. System Components Overview

```
+-----------------------------------------------+
|                SKYLITE / SKYNET               |
+-----------------------------------------------+
| Web Server Layer (IIS / Kestrel)              |
| Middleware (IHandler / SkyLiteHandler)        |
| Page Resolver                                 |
| WebPage Lifecycle Engine                      |
| HtmlDocument Builder                          |
| ApiRequest / ApiResponse Processor            |
| Translation Engine                            |
| JSCore Client Engine                          |
+-----------------------------------------------+
```

4. Request Processing Model

4.1 GET Request Flow

```
Browser → GET /PageName
    ↓
Framework Middleware
    ↓
Page Resolver (Reflection)
    ↓
WebPage Instance
    ↓
Lifecycle Execution
    ↓
HTML Output
```

4.2 POST Request Flow

```
Browser → POST ApiRequest
    ↓
Framework Middleware
    ↓
Page Resolver
    ↓
Invoke Method
    ↓
ApiResponse
    ↓
JSON Output
    ↓
JSCore Executes Commands
```

5. Page Resolution Engine

5.1 Reflection-Based Resolution

The framework resolves pages using:

```
Assembly.GetType(className)
```

5.2 Naming Requirements

- Class names must be unique.
- Namespaces may be used to avoid collisions.
- Folder structure is irrelevant.

5.3 Recommended Naming Pattern

```
HR_UserProfile
CRM_CustomerEdit
INV_ItemList
```

6. WebPage Lifecycle (Deep Dive)

6.1 Lifecycle Overview

```
OnLoad()
OnInitialized()
OnBeforeRender()
OnAfterRender()
```

6.2 OnLoad

- Load template file (HTML/TXT)
- Initialize HtmlDocument
- Prepare base structure

6.3 OnInitialized

- Add controls
- Modify document
- Apply security rules
- Append dynamic content

6.4 OnBeforeRender

- Inject scripts
- Finalize HTML
- Apply translation

6.5 OnAfterRender

- Final modifications

7. HtmlDocument Architecture

```
+---------------------------+
|       HtmlDocument        |
+---------------------------+
| HtmlBodyText              |
| HtmlBodyAddOn             |
| HeadContent               |
| ScriptInjection           |
| MetaTags                  |
+---------------------------+
```

7.1 HtmlBodyText

Loaded from template file.

7.2 HtmlBodyAddOn

Dynamic content appended by developer.

7.3 HeadContent

Scripts, styles, meta tags.

8. ApiRequest / ApiResponse Model

8.1 ApiRequest Structure

```
PageName
MethodName
Parameters
```

8.2 ApiResponse Structure

```
Commands[]
  - SetValue
  - UpdateHtml
  - ShowDialog
  - Notify
  - Redirect
  - RefreshGrid
```

9. JSCore Execution Engine

JSCore is responsible for:

- Sending ApiRequest
- Receiving ApiResponse
- Executing commands
- Updating DOM

9.1 Command Execution Flow

```
ApiResponse JSON
    ↓
JSCore Command Engine
    ↓
DOM Manipulation
```

10. Translation System

10.1 GET Translation

Applied to HtmlDocument.

10.2 POST Translation

Applied to ApiResponse.

10.3 Language Source

- Browser language
- Internal dictionary

11. Security Architecture

11.1 Security Hooks

- OnInitialized
- OnRequest
- OnBeforeRender

11.2 Security Flow

```
Request
    ↓
Security Checks
    ↓
Allow / Deny
```

12. Error Handling & Diagnostics

12.1 Error Types

- Page resolution errors
- Method invocation errors
- Template loading errors
- ApiResponse serialization errors

12.2 Logging

Framework logs:

- Exceptions
- Request metadata
- Page/method names

13. Performance Characteristics

13.1 Server-Side Rendering

Fast initial load.

13.2 Lightweight JSON

Efficient ApiResponse.

13.3 No Virtual DOM

Direct DOM manipulation.

14. Naming Conventions & Module Structure

14.1 Recommended Pattern

```
Module_PageName
```

14.2 Examples

```
HR_UserProfile
CRM_CustomerList
FIN_InvoiceEdit
```

15. Template System

15.1 Template Loading

Framework checks for:

```
PageName.html
PageName.txt
```

15.2 Dynamic Content

Added via:

```
HtmlBodyAddOn
```

16. Middleware Architecture (SKYNET)

```
builder.Services.AddHttpContextAccessor();
app.UseMiddleware<IHandler>();
app.UseStaticHttpCurrent();
app.Run();
```

16.1 IHandler Responsibilities

- Intercept requests
- Detect GET/POST
- Resolve page
- Execute lifecycle
- Serialize ApiResponse

17. Static Environment Compatibility

SKYNET provides:

- HttpCurrent.Request
- HttpCurrent.Response
- HttpCurrent.MapPath
- HttpCurrent.WebRootPath

This ensures SKYLITE compatibility.

18. Database Interaction Patterns

18.1 Typical Pattern

- Use ADO.NET or custom DAL
- Execute stored procedures
- Map results to objects

18.2 Connection Handling

Connections should be short-lived.

19. File Upload/Download Pipeline

19.1 Upload Flow

- JSCore sends file
- Framework receives stream
- Developer processes file

19.2 Download Flow

- Developer returns file stream
- Framework sets headers

20. Grid & Form Architecture

20.1 Grid Behavior

- Server-driven data loading
- ApiResponse refresh commands

20.2 Form Behavior

- Field binding via JSCore
- SetValue commands update UI

21. Developer Guidelines

21.1 Keep Pages Small

Each page should focus on one task.

21.2 Use Naming Prefixes

Avoid collisions.

21.3 Use Lifecycle Properly

Do not overload OnLoad.

22. Extensibility Model

22.1 Custom Controls

Developers can create reusable components.

22.2 Custom Commands

Extend ApiResponse with new commands.

23. Appendix A – Architecture Diagrams

(Already included in Document 1.)

24. Appendix B – Lifecycle Sequence Charts

24.1 GET Sequence

```
OnLoad → OnInitialized → OnBeforeRender → OnAfterRender
```

24.2 POST Sequence

```
Resolve Page → Invoke Method → ApiResponse → JSCore
```

25. Appendix C – Example Page Implementations

25.1 Basic Page

```
public class HR_UserProfile : WebPage
{
    public override void OnLoad()
    {
        // Load template
    }

    public override void OnInitialized()
    {
        // Add dynamic content
    }

    public ApiResponse Save()
    {
        return ApiResponse.Success("Saved");
    }
}
```